TP 1 : Programmation Python

Boucles, listes, dictionnaires et fonctions

Consignes. Répondez directement sur la feuille. Pour les questions « À vous de coder », écrivez du code Python 3 valide (attention à l'indentation). Pour les traces d'exécution, donnez exactement l'affichage produit par `print`.

Exercice 1 : Les boucles

▶ 1.1 Boucle for

Donnez l'affichage produit par le code suivant :

```
for i in range(1, 6):
    print(i * i)
```

▶ 1.2 La fonction range

Complétez le tableau avec les valeurs successivement prises par la variable de boucle.

Instruction	Valeurs générées
`range(5)`	
`range(2, 8)`	
`range(1, 10, 4)`	
`range(10, 0, -3)`	

▶ 1.3 Boucle while

```
total = 0
n = 1
while total < 20:
    total += n
    n += 1
print(total, n)
```

a) Combien de fois le corps de la boucle est-il exécuté ?

.....

b) Quel est l'affichage final ?

.....

▶ 1.4 break et continue

```python
for i in range(1, 10):
    if i % 2 == 0:
        continue
    if i > 6:
        break
    print(i, end=" ")
```

a) Quel est l'affichage produit ?

.....

b) Quel est le rôle de `continue` ? Quel est celui de `break` ?

.....

.....

▶ 1.5 À vous de coder : table de multiplication

Écrivez une boucle qui affiche la table de multiplication de 7, de 7×1 à 7×10, sous la forme `7 x 1 = 7`.

▶ 1.6 À vous de coder : somme

Écrivez un programme qui calcule et affiche la somme des entiers pairs de 1 à 100.

Exercice 2 : Les listes

On considère la liste suivante :

```
notes = [12, 15, 8, 17, 10]
```

▶ 2.1 Indices et tranches

Donnez la valeur de chaque expression.

Expression	Valeur
`notes[0]`	
`notes[-1]`	
`notes[1:3]`	
`notes[::-1]`	
`len(notes)`	

▶ 2.2 Méthodes de liste

À partir de la liste `notes` initiale, donnez l'affichage produit par :

```
notes.append(14)
notes.remove(8)
notes.insert(1, 9)
notes.sort()
print(notes)
```

▶ 2.3 À vous de coder : parcours

Sans utiliser `sum()` ni `max()`, écrivez un programme qui parcourt la liste `notes` initiale avec une boucle `for` puis affiche sa moyenne et sa plus grande valeur.

Exercice 3 : Les dictionnaires

On considère le dictionnaire suivant :

```
etudiant = {"nom": "Amine", "age": 20, "notes": [12, 15, 8]}
```

▶ 3.1 Accès aux valeurs

Expression	Valeur
`etudiant["nom"]`	
`etudiant["notes"][1]`	
`etudiant.get("ville", "inconnue")`	
`len(etudiant)`	
`"age"in etudiant`	

▶ 3.2 Modification

À partir du dictionnaire initial, donnez l'affichage produit par :

```
etudiant["age"] = 21
etudiant["ville"] = "Tlemcen"
del etudiant["notes"]
print(etudiant)
```

▶ 3.3 Parcours

```
stock = {"pomme": 5, "banane": 0, "orange": 3}
for fruit, qte in stock.items():
    if qte > 0:
        print(fruit, qte)
```

a) Quel est l'affichage produit ?

b) Que renvoie `stock.items()` ?

.....

▶ 3.4 À vous de coder : fréquences

Soit `mot = "banane"`. Écrivez un programme qui construit un dictionnaire `freq` contenant le nombre d'occurrences de chaque lettre, puis l'affiche.

▶ 3.5 Clés de dictionnaire

Peut-on utiliser une liste comme clé d'un dictionnaire ? Et un tuple ? Justifiez.

. . . . . .

. . . . . .

Exercice 4 : Les fonctions

On considère les fonctions suivantes :

```python
def carre(x):
    return x * x

def saluer(nom, politesse="Bonjour"):
    print(politesse + " " + nom)
```

▶ 4.1 Appels

Complétez le tableau.

Appel	Résultat (valeur renvoyée ou affichage)
`carre(4)`	
`saluer("Sara")`	
`saluer("Sara", "Salut")`	
`print(saluer("Sara"))`	

▶ 4.2 print ou return ?

Quelle est la différence entre `print` et `return` ? Pourquoi le dernier appel du tableau affiche-t-il une valeur supplémentaire ?

.....

.....

▶ 4.3 Portée des variables

```python
x = 10

def f():
    x = 5
    return x + 1

print(f(), x)
```

a) Quel est l'affichage produit ?

.....

b) Pourquoi la variable `x` globale n'est-elle pas modifiée ?

.....

.....

▶ 4.4 Arguments mutables

```python
def ajouter(liste, valeur):
    liste.append(valeur)

l = [1, 2]
ajouter(l, 3)
print(l)
```

a) Quel est l'affichage produit ?

.....

b) La fonction `ajouter` ne contient aucun `return`. Pourquoi la liste `l` est-elle pourtant modifiée ?

.....

.....

▶ 4.5 À vous de coder

a) Écrivez une fonction `est_pair(n)` qui renvoie `True` si `n` est pair, `False` sinon.

b) Écrivez une fonction `compter_mots(phrase)` qui renvoie un dictionnaire donnant le nombre d'occurrences de chaque mot de la phrase. Par exemple, `compter_mots("a b a")` renvoie `{'a': 2, 'b': 1}`. (Indice : `phrase.split()`.)

Exercice 5 : Mini-projet, gestion de notes

On considère le programme suivant, qui combine boucles, listes, dictionnaires et fonctions :

```python
def moyenne(notes):
    if len(notes) == 0:
        return 0.0
    total = 0
    for n in notes:
        total += n
    return total / len(notes)

def meilleur(classe):
    best = None
    best_moy = -1
    for nom, notes in classe.items():
        m = moyenne(notes)
        if m > best_moy:
            best_moy = m
            best = nom
    return best

classe = {"Amine": [12, 15, 9], "Sara": [16, 14, 18], "Yacine": []}
for nom in classe:
    print(nom, round(moyenne(classe[nom]), 1))
print("Meilleur :", meilleur(classe))
```

▶ 5.1 Trace d'exécution

Donnez, ligne par ligne, l'affichage produit par ce programme.

▶ 5.2 Cas particulier

Pourquoi la fonction `moyenne` teste-t-elle `len(notes)== 0` ? Que se passerait-il sans ce test pour l'étudiant `"Yacine"` ?

.....

.....

▶ 5.3 À vous de coder : ajout d'une note

Écrivez une fonction `ajouter_note(classe, nom, note)` qui ajoute `note` à la liste de l'étudiant `nom`. Si l'étudiant n'existe pas encore dans `classe`, il doit être créé.

▶ 5.4 À vous de coder : étudiants admis

Écrivez une fonction `admis(classe)` qui renvoie la liste des noms des étudiants dont la moyenne est supérieure ou égale à 10. Que renvoie `admis(classe)` pour le dictionnaire de l'énoncé ?

CORRECTION : TP 1

Exercice 1 : Les boucles

▶ 1.1 Boucle for

Correction

```
1
4
9
16
25
```

La boucle parcourt $i = 1, 2, 3, 4, 5$ (la borne 6 est exclue) et affiche le carré de chaque valeur.

▶ 1.2 La fonction range

Correction

`range(5)`	0, 1, 2, 3, 4
`range(2, 8)`	2, 3, 4, 5, 6, 7
`range(1, 10, 4)`	1, 5, 9
`range(10, 0, -3)`	10, 7, 4, 1

La borne de fin est toujours **exclue**. Le troisième argument est le pas (négatif pour compter à rebours).

▶ 1.3 Boucle while

Correction

a) **6 itérations** : `total` vaut successivement 1, 3, 6, 10, 15, 21.

b) L'affichage est `21 7`. La condition `total < 20` devient fausse lorsque `total` atteint 21 ; `n` vaut alors 7.

▶ 1.4 break et continue

Correction

a) L'affichage est `1 3 5` : les nombres pairs sont ignorés par `continue`, et la boucle s'arrête dès que $i = 7$ grâce à `break`.

b) `continue` passe directement à l'itération suivante ; `break` quitte complètement la boucle.

▶ 1.5 Table de multiplication

Correction

```python
for i in range(1, 11):
    print("7 x", i, "=", 7 * i)
```

▶ 1.6 Somme

Correction

```python
total = 0
for i in range(2, 101, 2):
    total += i
print(total) # 2550
```

Variante : `for i in range(1, 101)` avec le test `if i % 2 == 0`.

Exercice 2 : Les listes

▶ 2.1 Indices et tranches

Correction

`notes[0]`	12
`notes[-1]`	10 (dernier élément)
`notes[1:3]`	`[15, 8]` (indices 1 et 2, le 3 est exclu)
`notes[::-1]`	`[10, 17, 8, 15, 12]` (liste inversée)
`len(notes)`	5

▶ 2.2 Méthodes de liste

Correction

`[9, 10, 12, 14, 15, 17]`

Étapes : `[12, 15, 8, 17, 10, 14]` après `append`, `[12, 15, 17, 10, 14]` après `remove(8)`, `[12, 9, 15, 17, 10, 14]` après `insert(1, 9)`, puis tri croissant.

▶ 2.3 Parcours

Correction

```python
notes = [12, 15, 8, 17, 10]
total = 0
maximum = notes[0]
for n in notes:
    total += n
    if n > maximum:
        maximum = n
print(total / len(notes)) # 12.4
print(maximum) # 17
```

Exercice 3 : Les dictionnaires

▶ 3.1 Accès aux valeurs

Correction

`etudiant["nom"]`	`'Amine'`
`etudiant["notes"][1]`	15
`etudiant.get("ville", "inconnue")`	`'inconnue'` (clé absente : valeur par défaut)
`len(etudiant)`	3
`"age"in etudiant`	`True` (le test porte sur les clés)

▶ 3.2 Modification

Correction

```
{'nom': 'Amine', 'age': 21, 'ville': 'Tlemcen'}
```

`age` est modifiée, `ville` est ajoutée (nouvelle clé) et `notes` est supprimée.

▶ 3.3 Parcours

Correction

a)
```
pomme 5
orange 3
```

La banane est ignorée car sa quantité est nulle.

b) `items()` renvoie l'ensemble des paires (clé, valeur) du dictionnaire ; la boucle les dépaquette dans `fruit` et `qte`.

▶ 3.4 Fréquences

Correction

```python
mot = "banane"
freq = {}
for c in mot:
    freq[c] = freq.get(c, 0) + 1
print(freq) # {'b': 1, 'a': 2, 'n': 2, 'e': 1}
```

▶ 3.5 Clés de dictionnaire

Correction

Une **liste** ne peut pas être une clé : elle est modifiable (mutable), donc non « hachable » (`TypeError: unhashable type: 'list'`). Un **tuple** d'éléments immuables est accepté, car il est immuable. Les chaînes et les nombres sont aussi des clés valides.

Exercice 4 : Les fonctions

▶ 4.1 Appels

Correction

`carre(4)`	16 (valeur renvoyée)
`saluer("Sara")`	affiche `Bonjour Sara`
`saluer("Sara", "Salut")`	affiche `Salut Sara`
`print(saluer("Sara"))`	affiche `Bonjour Sara` puis `None`

▶ 4.2 print ou return ?

Correction

`print` **affiche** une valeur à l'écran, mais ne la rend pas utilisable par le programme. `return` **renvoie** une valeur à l'appelant (qui peut la stocker ou la réutiliser) et termine la fonction. Une fonction sans `return` renvoie `None` : c'est ce que le dernier `print` affiche.

▶ 4.3 Portée des variables

Correction

a) L'affichage est `6 10`.

b) L'affectation `x = 5` dans la fonction crée une variable **locale**, distincte de la variable globale `x`. La globale garde donc la valeur 10.

▶ 4.4 Arguments mutables

Correction

a) L'affichage est `[1, 2, 3]`.

b) La fonction reçoit une **référence** vers la même liste que celle de l'appelant : elle la modifie directement, sans en créer de copie. Une liste étant modifiable (mutable), le changement est visible après l'appel, sans `return`.

▶ 4.5 À vous de coder

Correction

```python
def est_pair(n):
    return n % 2 == 0

def compter_mots(phrase):
    compteur = {}
    for mot in phrase.split():
        compteur[mot] = compteur.get(mot, 0) + 1
    return compteur
```

Exercice 5 : Mini-projet, gestion de notes

▶ 5.1 Trace d'exécution

Correction

```
Amine 12.0
Sara 16.0
Yacine 0.0
Meilleur : Sara
```

Moyennes : Amine = 36/3 = 12,0 ; Sara = 48/3 = 16,0 ; Yacine n'a aucune note, donc `moyenne` renvoie `0.0`.

▶ 5.2 Cas particulier

Correction

Sans ce test, le calcul `total / len(notes)` pour `"Yacine"` diviserait par 0 et provoquerait une erreur `ZeroDivisionError`. Le test traite le cas de la liste vide avant de faire la division.

▶ 5.3 Ajout d'une note

Correction

```
def ajouter_note(classe, nom, note):
    if nom not in classe:
        classe[nom] = []
    classe[nom].append(note)
```

Le dictionnaire est modifié directement (argument mutable) : la fonction n'a pas besoin de `return`.

▶ 5.4 Étudiants admis

Correction

```
def admis(classe):
    resultat = []
    for nom, notes in classe.items():
        if moyenne(notes) >= 10:
            resultat.append(nom)
    return resultat
```

Pour le dictionnaire de l'énoncé, le résultat est `['Amine', 'Sara']` : Yacine, avec une moyenne de 0.0, est refusé.